Arrays Part 1: Easy Problems

Use this header for all C++ snippets:

```
#include <bits/stdc++.h>
  using namespace std;
```

1. Largest Element in an Array

Problem

Given an array, find the largest element.

Brute Force: Sort

Sort the array and return the last element.

```
int largestBrute(vector<int> arr) {
      sort(arr.begin(), arr.end());
      return arr.back();
  }
```

Time Complexity: O(n log n)
Space Complexity: O(1), ignoring sorting recursion

Optimal: Linear Scan

Keep one variable for the maximum element.

```
int largestOptimal(vector<int>& arr) {
      int largest = arr[0];
      for (int i = 1; i < arr.size(); i++) {
          if (arr[i] > largest) largest = arr[i];
      }
      return largest;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

2. Second Largest Element without Sorting

Problem

Find the second largest distinct element. If it does not exist, return `-1`.

Brute Force: Sort

Sort, then move left from the second-last index until a smaller value is found.

```
int secondLargestBrute(vector<int> arr) {
      sort(arr.begin(), arr.end());
      int largest = arr.back();
      for (int i = arr.size() - 2; i >= 0; i--) {
          if (arr[i] != largest) return arr[i];
      }
      return -1;
  }
```

Time Complexity: O(n log n)
Space Complexity: O(1), ignoring sorting recursion

Better: Two Passes

First find the largest, then find the best value smaller than largest.

```
int secondLargestBetter(vector<int>& arr) {
      int largest = arr[0];
      for (int x : arr) largest = max(largest, x);

      int secondLargest = -1;
      for (int x : arr) {
          if (x < largest && x > secondLargest) secondLargest = x;
      }
      return secondLargest;
  }
```

Time Complexity: O(2n)
Space Complexity: O(1)

Optimal: One Pass

Update largest and second largest together.

```
int secondLargestOptimal(vector<int>& arr) {
      int largest = arr[0];
      int secondLargest = -1;

      for (int i = 1; i < arr.size(); i++) {
          if (arr[i] > largest) {
              secondLargest = largest;
              largest = arr[i];
          } else if (arr[i] < largest && arr[i] > secondLargest) {
              secondLargest = arr[i];
          }
      }
      return secondLargest;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

3. Check if the Array is Sorted

Problem

Check whether the array is sorted in non-decreasing order.

Brute Force: Check All Pairs

For every `i < j`, `arr[i]` must be less than or equal to `arr[j]`.

```
bool isSortedBrute(vector<int>& arr) {
      for (int i = 0; i < arr.size(); i++) {
          for (int j = i + 1; j < arr.size(); j++) {
              if (arr[i] > arr[j]) return false;
          }
      }
      return true;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)

Optimal: Adjacent Check

Only compare neighbouring elements.

```
bool isSortedOptimal(vector<int>& arr) {
      for (int i = 1; i < arr.size(); i++) {
          if (arr[i] < arr[i - 1]) return false;
      }
      return true;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

4. Remove Duplicates from Sorted Array

Problem

Given a sorted array, remove duplicates in-place and return the number of unique elements.

Brute Force: Set

Insert elements into a set, then copy unique values back.

```
int removeDuplicatesBrute(vector<int>& arr) {
      set<int> st;
      for (int x : arr) st.insert(x);

      int index = 0;
      for (int x : st) arr[index++] = x;
      return index;
  }
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Optimal: Two Pointers

Keep `i` at the last unique element and scan with `j`.

```
int removeDuplicatesOptimal(vector<int>& arr) {
      int i = 0;
      for (int j = 1; j < arr.size(); j++) {
          if (arr[j] != arr[i]) {
              i++;
              arr[i] = arr[j];
          }
      }
      return i + 1;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

5. Left Rotate an Array by One Place

Problem

Move the first element to the end and shift all other elements left by one.

Optimal

Store the first element, shift, then place it at the end.

```
void leftRotateByOne(vector<int>& arr) {
      int temp = arr[0];
      for (int i = 1; i < arr.size(); i++) {
          arr[i - 1] = arr[i];
      }
      arr[arr.size() - 1] = temp;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

6. Left Rotate an Array by D Places

Problem

Rotate the array left by `d` positions.

Brute Force: Rotate One by One

Perform one-left-rotation `d` times.

```
void leftRotateDBrute(vector<int>& arr, int d) {
      int n = arr.size();
      d = d % n;
      while (d--) {
          int temp = arr[0];
          for (int i = 1; i < n; i++) arr[i - 1] = arr[i];
          arr[n - 1] = temp;
      }
  }
```

Time Complexity: O(n × d)
Space Complexity: O(1)

Better: Temporary Array

Store the first `d` elements, shift the rest, then copy temp back.

```
void leftRotateDBetter(vector<int>& arr, int d) {
      int n = arr.size();
      d = d % n;

      vector<int> temp;
      for (int i = 0; i < d; i++) temp.push_back(arr[i]);

      for (int i = d; i < n; i++) arr[i - d] = arr[i];
      for (int i = n - d; i < n; i++) arr[i] = temp[i - (n - d)];
  }
```

Time Complexity: O(n)
Space Complexity: O(d)

Optimal: Reverse Technique

Reverse first `d`, reverse remaining `n-d`, then reverse the whole array.

```
void leftRotateDOptimal(vector<int>& arr, int d) {
      int n = arr.size();
      d = d % n;
      reverse(arr.begin(), arr.begin() + d);
      reverse(arr.begin() + d, arr.end());
      reverse(arr.begin(), arr.end());
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

7. Move Zeroes to End

Problem

Move all zeroes to the end while keeping non-zero elements in the same order.

Brute Force: Temporary Array

Store all non-zero elements, copy them back, then fill remaining places with zero.

```
void moveZeroesBrute(vector<int>& arr) {
      vector<int> temp;
      for (int x : arr) {
          if (x != 0) temp.push_back(x);
      }

      for (int i = 0; i < temp.size(); i++) arr[i] = temp[i];
      for (int i = temp.size(); i < arr.size(); i++) arr[i] = 0;
  }
```

Time Complexity: O(n)
Space Complexity: O(n)

Optimal: Two Pointers

Find the first zero, then swap it with later non-zero elements.

```
void moveZeroesOptimal(vector<int>& arr) {
      int j = -1;
      for (int i = 0; i < arr.size(); i++) {
          if (arr[i] == 0) {
              j = i;
              break;
          }
      }

      if (j == -1) return;

      for (int i = j + 1; i < arr.size(); i++) {
          if (arr[i] != 0) {
              swap(arr[i], arr[j]);
              j++;
          }
      }
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

8. Linear Search

Problem

Find the index of a target element. If not present, return `-1`.

Optimal

Scan from left to right.

```
int linearSearch(vector<int>& arr, int target) {
      for (int i = 0; i < arr.size(); i++) {
          if (arr[i] == target) return i;
      }
      return -1;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

9. Union of Two Sorted Arrays

Problem

Given two sorted arrays, return their sorted union without duplicates.

Brute Force: Set

Insert all elements into a set.

```
vector<int> unionBrute(vector<int>& a, vector<int>& b) {
      set<int> st;
      for (int x : a) st.insert(x);
      for (int x : b) st.insert(x);

      vector<int> ans;
      for (int x : st) ans.push_back(x);
      return ans;
  }
```

Time Complexity: O((n + m) log(n + m))
Space Complexity: O(n + m)

Optimal: Two Pointers

Move through both sorted arrays and push only new values.

```
vector<int> unionOptimal(vector<int>& a, vector<int>& b) {
      int i = 0, j = 0;
      vector<int> ans;

      while (i < a.size() && j < b.size()) {
          int value;
          if (a[i] <= b[j]) value = a[i++];
          else value = b[j++];

          if (ans.empty() || ans.back() != value) ans.push_back(value);
      }

      while (i < a.size()) {
          if (ans.empty() || ans.back() != a[i]) ans.push_back(a[i]);
          i++;
      }

      while (j < b.size()) {
          if (ans.empty() || ans.back() != b[j]) ans.push_back(b[j]);
          j++;
      }

      return ans;
  }
```

Time Complexity: O(n + m)
Space Complexity: O(n + m) for answer

9. Intersection of two sorted array

int n1=arr1.size(); int n2= arr2.size(); vector< int> ans; vector< int> vis(n2, 0); for(int i=0; i `for( int j=0; j<n2; j++){`

```
      if(arr1[i]==arr2[j]&& vis[j]==0){
          vis[j]=1;
          ans.push_back( arr[i]);
          break;
```

```
      }
        if(arr2[j]>arr1[i])break;
    }
```

}

// OPtimal; int i =0, j=0; vector ans; while(i `if( arr1[i]<arr2[j]){` `i++;` `}` `else if(arr2[j]<arr1[i]){` `j++;` `}` `else{` `ans.push_back(arr1[i]);` `i++;j++;` `}` } return ans;

10. Missing Number in an Array

Problem

Given `n-1` numbers from `1` to `n`, find the missing number.

Brute Force: Linear Search for Every Number

For each number from `1` to `n`, check if it exists.

```
int missingBrute(vector<int>& arr, int n) {
      for (int num = 1; num <= n; num++) {
          bool found = false;
          for (int x : arr) {
              if (x == num) {
                  found = true;
                  break;
              }
          }
          if (!found) return num;
      }
      return -1;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Better: Hashing

Mark every present number.

```
int missingBetter(vector<int>& arr, int n) {
      vector<int> hash(n + 1, 0);
      for (int x : arr) hash[x] = 1;

      for (int num = 1; num <= n; num++) {
          if (hash[num] == 0) return num;
      }
      return -1;
  }
```

Time Complexity: O(n)
Space Complexity: O(n)

Optimal: XOR

XOR all numbers from `1` to `n` and XOR all array elements. Pairs cancel out.

```
int missingOptimal(vector<int>& arr, int n) {
      int xr = 0;
      for (int num = 1; num <= n; num++) xr ^= num;
      for (int x : arr) xr ^= x;
      return xr;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

11. Maximum Consecutive Ones

Problem

Find the maximum number of consecutive `1`s in a binary array.

Optimal

Maintain current streak and best streak.

```
int maximumConsecutiveOnes(vector<int>& arr) {
      int count = 0;
      int maxi = 0;

      for (int x : arr) {
          if (x == 1) {
              count++;
              maxi = max(maxi, count);
          } else {
              count = 0;
          }
      }
      return maxi;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

12. Find the Number that Appears Once, Others Twice

Problem

Every element appears twice except one element. Find that single element.

Brute Force: Count Every Element

For each element, count its frequency.

```
int singleNumberBrute(vector<int>& arr) {
      for (int i = 0; i < arr.size(); i++) {
          int count = 0;
          for (int j = 0; j < arr.size(); j++) {
              if (arr[j] == arr[i]) count++;
          }
          if (count == 1) return arr[i];
      }
      return -1;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)

Better: Hash Map

Count frequency of every number.

```
int singleNumberBetter(vector<int>& arr) {
      unordered_map<int, int> freq;
      for (int x : arr) freq[x]++;

      for (auto it : freq) {
          if (it.second == 1) return it.first;
      }
      return -1;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

Optimal: XOR

Same numbers cancel because `x ^ x = 0`.

```
int singleNumberOptimal(vector<int>& arr) {
      int xr = 0;
      for (int x : arr) xr ^= x;
      return xr;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

13. Longest Subarray with Sum K: Positives

Problem

Find the longest subarray with sum `k` when all elements are positive.

Brute Force: Generate Subarrays

Check every subarray sum.

```
int longestSubarrayPositiveBrute(vector<int>& arr, long long k) {
      int maxLen = 0;
      for (int i = 0; i < arr.size(); i++) {
          long long sum = 0;
          for (int j = i; j < arr.size(); j++) {
              sum += arr[j];
              if (sum == k) maxLen = max(maxLen, j - i + 1);
          }
      }
      return maxLen;
  }
```

Time Complexity: O(n^2)
Space Complexity: O(1)

Better: Prefix Sum Hash Map

Works for positives and negatives also.

```
int longestSubarrayPositiveBetter(vector<int>& arr, long long k) {
      unordered_map<long long, int> firstIndex;
      long long sum = 0;
      int maxLen = 0;

      for (int i = 0; i < arr.size(); i++) {
          sum += arr[i];

          if (sum == k) maxLen = max(maxLen, i + 1);

          long long rem = sum - k;
          if (firstIndex.count(rem)) {
              maxLen = max(maxLen, i - firstIndex[rem]);
          }

          if (!firstIndex.count(sum)) firstIndex[sum] = i;
      }
      return maxLen;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

Optimal: Two Pointers

Since all numbers are positive, shrinking the left pointer always reduces the sum.

```
int longestSubarrayPositiveOptimal(vector<int>& arr, long long k) {
      int left = 0;
      long long sum = 0;
      int maxLen = 0;

      for (int right = 0; right < arr.size(); right++) {
          sum += arr[right];

          while (left <= right && sum > k) {
              sum -= arr[left];
              left++;
          }

          if (sum == k) maxLen = max(maxLen, right - left + 1);
      }
      return maxLen;
  }
```

Time Complexity: O(2n), effectively O(n)
Space Complexity: O(1)

14. Longest Subarray with Sum K: Positives and Negatives

Problem

Find the longest subarray with sum `k` when the array can contain positive, negative, and zero values.

Brute Force: Generate Subarrays

```
int longestSubarrayAnyBrute(vector<int>& arr, long long k) {
      int maxLen = 0;
      for (int i = 0; i < arr.size(); i++) {
          long long sum = 0;
          for (int j = i; j < arr.size(); j++) {
              sum += arr[j];
              if (sum == k) maxLen = max(maxLen, j - i + 1);
          }
      }
      return maxLen;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Optimal: Prefix Sum Hash Map

Two pointers fails with negative numbers. Store the first index where each prefix sum appears.

```
int longestSubarrayAnyOptimal(vector<int>& arr, long long k) {
      unordered_map<long long, int> firstIndex;
      long long sum = 0;
      int maxLen = 0;

      for (int i = 0; i < arr.size(); i++) {
          sum += arr[i];

          if (sum == k) maxLen = max(maxLen, i + 1);

          long long rem = sum - k;
          if (firstIndex.count(rem)) {
              maxLen = max(maxLen, i - firstIndex[rem]);
          }

          if (!firstIndex.count(sum)) firstIndex[sum] = i;
      }
      return maxLen;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

number of subarrya iwth sum ==k

#include using namespace std;

```
int countSubarrays(vector<int>& arr, int k) {
      unordered_map<long long, int> mp;

      long long sum = 0;
      int ans = 0;

      mp[0] = 1;//this is for when we take array from 0 to i

      for (int x : arr) {
          sum += x;

          if (mp.count(sum - k))
              ans += mp[sum - k];

          mp[sum]++;
      }

      return ans;
  }
```

this code will work for positve negative and zeroes all

Arrays Part 2: Medium Problems

Use this header for all C++ snippets:

```
#include <bits/stdc++.h>
  using namespace std;
```

15. Two Sum

Problem

Given an array and a target, find two indices whose values add up to the target.

Brute Force 1: Check All Pairs

```
vector<int> twoSumBrute(vector<int>& arr, int target) {
      for (int i = 0; i < arr.size(); i++) {
          for (int j = 0; j < arr.size(); j++) {
                if(i==j)continue;//because we cannot take same index element twice
              if (arr[i] + arr[j] == target) return {i, j};
          }
      }
      return {-1, -1};
  }
```

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Brute Force 2: Check All Pairs

```
vector<int> twoSumBrute(vector<int>& arr, int target) {
      for (int i = 0; i < arr.size(); i++) {
          for (int j = i + 1; j < arr.size(); j++) {
              if (arr[i] + arr[j] == target) return {i, j};
          }
      }
      return {-1, -1};
  }
```

Time Complexity: slightly less than$O(n^2)$
Space Complexity: O(1)

Better: Hash Map

For each element, check if `target - arr[i]` was seen before.

```
vector<int> twoSumBetter(vector<int>& arr, int target) {
      unordered_map<int, int> mp;

      for (int i = 0; i < arr.size(); i++) {
          int need = target - arr[i];
          if (mp.count(need)) return {mp[need], i};
          mp[arr[i]] = i;
      }
      return {-1, -1};
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

Optimal for Yes/No Variant: Sort + Two Pointers

If only `YES/NO` is needed, sorting avoids hash space.

```
bool twoSumOptimalYesNo(vector<int> arr, int target) {
      sort(arr.begin(), arr.end());
      int left = 0, right = arr.size() - 1;

      while (left < right) {
          int sum = arr[left] + arr[right];
          if (sum == target) return true;
          if (sum < target) left++;
          else right--;
      }
      return false;
  }
```

Time Complexity: O(n log n)
Space Complexity: O(1), ignoring sorting recursion

Optimal for Yes/No with index of two values Variant: Sort + Two Pointers

If only `YES/NO` is needed, sorting avoids hash space.

```
pair< int ,int> twoSumOptimalYesNo(vector<int> arr, int target) {
      vector<pair< int ,int>>arr1;
      for( int i=0; i<n ;i++){
           arr1.push_back({arr[i],i});
      }
      sort(arr1.begin(), arr1.end());
      int left = 0, right = arr1.size() - 1;

      while (left < right) {
          int sum = arr1[left].first + arr1[right].   first;
          if (sum == target) return  {arr1[left].second , arr1[right].second};
          if (sum < target) left++;
          else right--;
      }
      return {-1, -1};
  }
```

Time Complexity: O(n log n)
Space Complexity: O(1), ignoring sorting recursion

Count all index pairs (i, j) where i < j (Most Common)

Use a hash map of frequencies.

```
int countPairs(vector<int>& nums, int target) {
      unordered_map<int, int> freq;
      int ans = 0;

      for (int x : nums) {
          int need = target - x;

          if (freq.count(need))
              ans += freq[need];

          freq[x]++;
      }

      return ans;
  }
```

16. Sort an Array of 0s, 1s and 2s

Problem

Sort an array containing only `0`, `1`, and `2`.

Brute Force: Sorting

```
void sortColorsBrute(vector<int>& arr) {
      sort(arr.begin(), arr.end());
  }
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Better: Count 0s, 1s and 2s

```
void sortColorsBetter(vector<int>& arr) {
      int count0 = 0, count1 = 0, count2 = 0;
      for (int x : arr) {
          if (x == 0) count0++;
          else if (x == 1) count1++;
          else count2++;
      }

      int index = 0;
      while (count0--) arr[index++] = 0;
      while (count1--) arr[index++] = 1;
      while (count2--) arr[index++] = 2;
  }
```

Time Complexity: O(2n)
Space Complexity: O(1)

Optimal: Dutch National Flag

Maintain three regions: `0`s, `1`s, and unknown values. everything from 0 to low-1 will store 0's everything from low to mid -1 will store 1 everything from high+1 to end will store 2's from 0 to mid-1 is sorted from high+1 to n-1 is sorted unsoted portion is mid to high isi pr hume focus krna hai

```
void sortColorsOptimal(vector<int>& arr) {
      int low = 0, mid = 0, high = arr.size() - 1;

      while (mid <= high) {
          if (arr[mid] == 0) {
              swap(arr[low], arr[mid]);
              low++;
              mid++;
          } else if (arr[mid] == 1) {
              mid++;
          } else {
              swap(arr[mid], arr[high]);
              high--;
          }
      }
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

17. Majority Element Greater than n/2

Problem

Find the element that appears more than `n/2` times. If no such element exists, return `-1`.

Brute Force: Count Each Element

```
int majorityN2Brute(vector<int>& arr) {
      int n = arr.size();
      for (int i = 0; i < n; i++) {
          int count = 0;
          for (int j = 0; j < n; j++) {
              if (arr[j] == arr[i]) count++;
          }
          if (count > n / 2) return arr[i];
      }
      return -1;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Better: Hash Map

```
int majorityN2Better(vector<int>& arr) {
      unordered_map<int, int> freq;
      for (int x : arr) {
          freq[x]++;
          if (freq[x] > arr.size() / 2) return x;
      }
      return -1;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

Optimal: Moore's Voting Algorithm

Cancel different elements. Verify the candidate at the end.

```
int majorityN2Optimal(vector<int>& arr) {
      int candidate = 0;
      int count = 0;

      for (int x : arr) {
          if (count == 0) {
              candidate = x;
              count = 1;
          } else if (x == candidate) {
              count++;
          } else {
              count--;
          }
      }
      //if there exists a majority elememnt it would be condidae so check for condidate
  frequency

      int freq = 0;
      for (int x : arr) {
          if (x == candidate) freq++;
      }
      return freq > arr.size() / 2 ? candidate : -1;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

18. Maximum Subarray Sum

Problem

Find the maximum possible sum of a contiguous subarray.

Brute Force: All Subarrays

```
long long maxSubarrayBrute(vector<int>& arr) {
      long long maxi = LLONG_MIN;

      for (int i = 0; i < arr.size(); i++) {
          long long sum = 0;
          for (int j = i; j < arr.size(); j++) {
              sum += arr[j];
              maxi = max(maxi, sum);
          }
      }
      return maxi;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Optimal: Kadane's Algorithm

If current sum becomes negative, drop it and start fresh.

```
long long maxSubarrayOptimal(vector<int>& arr) {
      long long sum = 0;
      long long maxi = LLONG_MIN;

      for (int x : arr) {
          sum += x;
          maxi = max(maxi, sum);

          if (sum < 0) sum = 0;
      }
      return maxi;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

19. Print Subarray with Maximum Subarray Sum

Problem

Print or return the subarray that gives the maximum subarray sum.

Brute Force: Track Best Range

```
vector<int> printMaxSubarrayBrute(vector<int>& arr) {
      long long best = LLONG_MIN;
      int start = 0, end = 0;

      for (int i = 0; i < arr.size(); i++) {
          long long sum = 0;
          for (int j = i; j < arr.size(); j++) {
              sum += arr[j];
              if (sum > best) {
                  best = sum;
                  start = i;
                  end = j;
              }
          }
      }

      vector<int> ans;
      for (int i = start; i <= end; i++) ans.push_back(arr[i]);
      return ans;
  }
```

Time Complexity: O(n²)
Space Complexity: O(length of answer)

Optimal: Kadane with Start and End

```
vector<int> printMaxSubarrayOptimal(vector<int>& arr) {
      long long sum = 0;
      long long best = LLONG_MIN;
      int tempStart = 0, start = 0, end = 0;

      for (int i = 0; i < arr.size(); i++) {
          if (sum == 0) tempStart = i;

          sum += arr[i];

          if (sum > best) {
              best = sum;
              start = tempStart;
              end = i;
          }

          if (sum < 0) sum = 0;
      }

      vector<int> ans;
      for (int i = start; i <= end; i++) ans.push_back(arr[i]);
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(length of answer)

20. Stock Buy and Sell

Problem

Given stock prices, choose one day to buy and one later day to sell for maximum profit.

Brute Force: Check Every Buy-Sell Pair

```
int stockProfitBrute(vector<int>& prices) {
      int profit = 0;
      for (int buy = 0; buy < prices.size(); buy++) {
          for (int sell = buy + 1; sell < prices.size(); sell++) {
              profit = max(profit, prices[sell] - prices[buy]);
          }
      }
      return profit;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Optimal: Track Minimum Price

```
int stockProfitOptimal(vector<int>& prices) {
      int mini = prices[0];
      int profit = 0;

      for (int i = 1; i < prices.size(); i++) {
          int cost = prices[i] - mini;
          profit = max(profit, cost);
          mini = min(mini, prices[i]);
      }
      return profit;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

21. Rearrange Array in Alternating Positive and Negative Items

Problem

Rearrange the array so positive and negative numbers appear alternately.

Brute Force: Separate Positives and Negatives

Works when positives and negatives are equal in count.

```
vector<int> rearrangeEqualBrute(vector<int>& arr) {
      vector<int> pos, neg;
      for (int x : arr) {
          if (x > 0) pos.push_back(x);
          else neg.push_back(x);
      }

      for (int i = 0; i < arr.size() / 2; i++) {
          arr[2 * i] = pos[i];
          arr[2 * i + 1] = neg[i];
      }
      return arr;
  }
```

Time Complexity: O(n)
Space Complexity: O(n)

Optimal for Equal Counts: Direct Placement

```
vector<int> rearrangeEqualOptimal(vector<int>& arr) {
      vector<int> ans(arr.size());
      int posIndex = 0;
      int negIndex = 1;

      for (int x : arr) {
          if (x > 0) {
              ans[posIndex] = x;
              posIndex += 2;
          } else {
              ans[negIndex] = x;
              negIndex += 2;
          }
      }
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(n)

Variant: Unequal Counts

```
vector<int> rearrangeUnequal(vector<int>& arr) {
      vector<int> pos, neg;
      for (int x : arr) {
          if (x > 0) pos.push_back(x);
          else neg.push_back(x);
      }

      vector<int> ans;
      int i = 0, j = 0;
      while (i < pos.size() && j < neg.size()) {
          ans.push_back(pos[i++]);
          ans.push_back(neg[j++]);
      }

      while (i < pos.size()) ans.push_back(pos[i++]);
      while (j < neg.size()) ans.push_back(neg[j++]);
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(n)

22. Next Permutation

Problem

Find the next lexicographically greater permutation. If it does not exist, return the smallest permutation.

Brute Force: Generate All Permutations

Generate all permutations, sort them, and pick the next one. This is only theoretical.

```
void generatePermutations(vector<int>& arr, int index, vector<vector<int>>& all) {
      if (index == arr.size()) {
          all.push_back(arr);
          return;
      }

      for (int i = index; i < arr.size(); i++) {
          swap(arr[index], arr[i]);
          generatePermutations(arr, index + 1, all);
          swap(arr[index], arr[i]);
      }
  }
```

Time Complexity: O(n! × n)
Space Complexity: O(n!)

Better: STL

```
void nextPermutationBetter(vector<int>& arr) {
      next_permutation(arr.begin(), arr.end());
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

Optimal: Manual Pivot Method

```
void nextPermutationOptimal(vector<int>& arr) {
      int n = arr.size();
      int index = -1;

      //finding the first breakpoint   from  back index i st a[i]<a[i+1];

      for (int i = n - 2; i >= 0; i--) {
          if (arr[i] < arr[i + 1]) {
              index = i;//breakpoint
              break;
          }
      }

      if (index == -1) {//agr breakpoint nhi h to first perpermulation return kr do
          reverse(arr.begin(), arr.end());
          return;
      }

      for (int i = n - 1; i > index; i--) {
          if (arr[i] > arr[index]) {
              swap(arr[i], arr[index]);//first element from back greater than a[index];
              break;
          }
      }

      reverse(arr.begin() + index + 1, arr.end());
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

23. Leaders in an Array

Problem

An element is a leader if every element to its right is smaller.

Brute Force: Check Right Side

```
vector<int> leadersBrute(vector<int>& arr) {
      vector<int> ans;

      for (int i = 0; i < arr.size(); i++) {
          bool leader = true;
          for (int j = i + 1; j < arr.size(); j++) {
              if (arr[j] > arr[i]) {
                  leader = false;
                  break;
              }
          }
          if (leader) ans.push_back(arr[i]);
      }
      return ans;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1), excluding answer

Optimal: Scan from Right

```
vector<int> leadersOptimal(vector<int>& arr) {
      vector<int> ans;
      int maxi = INT_MIN;

      for (int i = arr.size() - 1; i >= 0; i--) {
          if (arr[i] > maxi) {
              ans.push_back(arr[i]);
              maxi = arr[i];
          }
      }

      reverse(ans.begin(), ans.end());
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(1), excluding answer

24. Longest Consecutive Sequence

Problem

Find the length of the longest consecutive sequence in an unsorted array.

Brute Force: Linear Search Next Element

```
bool linearSearchValue(vector<int>& arr, int target) {
      for (int x : arr) {
          if (x == target) return true;
      }
      return false;
  }

  int longestConsecutiveBrute(vector<int>& arr) {
      int longest = 1;

      for (int x : arr) {
          int current = x;
          int count = 1;

          while (linearSearchValue(arr, current + 1)) {
              current++;
              count++;
          }
          longest = max(longest, count);
      }
      return longest;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Better: Sorting

```
int longestConsecutiveBetter(vector<int> arr) {
      sort(arr.begin(), arr.end());

      int longest = 1;
      int count = 0;
      int lastSmaller = INT_MIN;

      for (int x : arr) {
          if (x - 1 == lastSmaller) {
              count++;
              lastSmaller = x;
          } else if (x != lastSmaller) {
              count = 1;
              lastSmaller = x;
          }
          longest = max(longest, count);
      }
      return longest;
  }
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Optimal: Hash Set

Start counting only from numbers that do not have a previous value.

```
int longestConsecutiveOptimal(vector<int>& arr) {
      unordered_set<int> st(arr.begin(), arr.end());
      int longest = 0;

      for (int x : st) {
          if (!st.count(x - 1)) {
              int current = x;
              int count = 1;

              while (st.count(current + 1)) {
                  current++;
                  count++;
              }
              longest = max(longest, count);
          }
      }
      return longest;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

25. Set Matrix Zeroes

Problem

If any cell is `0`, make its whole row and column `0`.

Brute Force: Mark Rows and Columns

Mark cells with `-1` first, then convert them to zero. This works only if matrix values are non-negative.

```
void markRow(vector<vector<int>>& matrix, int row) {
      for (int col = 0; col < matrix[0].size(); col++) {
          if (matrix[row][col] != 0) matrix[row][col] = -1;
      }
  }

  void markCol(vector<vector<int>>& matrix, int col) {
      for (int row = 0; row < matrix.size(); row++) {
          if (matrix[row][col] != 0) matrix[row][col] = -1;
      }
  }

  void setZeroesBrute(vector<vector<int>>& matrix) {
      int n = matrix.size(), m = matrix[0].size();

      for (int row = 0; row < n; row++) {
          for (int col = 0; col < m; col++) {
              if (matrix[row][col] == 0) {
                  markRow(matrix, row);
                  markCol(matrix, col);
              }
          }
      }

      for (int row = 0; row < n; row++) {
          for (int col = 0; col < m; col++) {
              if (matrix[row][col] == -1) matrix[row][col] = 0;
          }
      }
  }
```

Time Complexity: $O((n \times m) \times (n + m))$
Space Complexity: $O(1)$

Better: Row and Column Arrays

```
void setZeroesBetter(vector<vector<int>>& matrix) {
      int n = matrix.size(), m = matrix[0].size();
      vector<int> row(n, 0), col(m, 0);

      for (int i = 0; i < n; i++) {
          for (int j = 0; j < m; j++) {
              if (matrix[i][j] == 0) {
                  row[i] = 1;
                  col[j] = 1;
              }
          }
      }

      for (int i = 0; i < n; i++) {
          for (int j = 0; j < m; j++) {
              if (row[i] || col[j]) matrix[i][j] = 0;
          }
      }
  }
```

Time Complexity: O(n × m)
Space Complexity: O(n + m)

Optimal: First Row and First Column as Markers

```
void setZeroesOptimal(vector<vector<int>>& matrix) {
      int n = matrix.size(), m = matrix[0].size();
      int col0 = 1;

      for (int i = 0; i < n; i++) {



          for (int j = 0; j < m; j++) {
              if (matrix[i][j] == 0) {
                  matrix[i][0] = 0;
                  if(j!==0){
                  matrix[0][j] = 0;
                  }
                  else{
                      col0=0;
                  }
              }
          }
      }

      for (int i = i; i <n; i++) {
          for (int j = 1; j <m; j++) {
              if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                  matrix[i][j] = 0;
              }
          }

      }
      if(mat[0][0]==0){
          for( int j=0; j<m ;j++){
               matrix[0][j]=0;
          }

      }
      if(col0==0){
           for( i=0; i<n ;i++){
               matrix[i][0]=0;
          }
```

```
        }
    }
```

Time Complexity: O(n × m)
Space Complexity: O(1)

26. Rotate Matrix by 90 Degrees

Problem

Rotate a square matrix by 90 degrees clockwise.

Brute Force: Extra Matrix

```
vector<vector<int>> rotateMatrixBrute(vector<vector<int>>& matrix) {
        int n = matrix.size();
        vector<vector<int>> rotated(n, vector<int>(n));
        // i j goes to j , n-1-i;;/ observation  but we put reverre order in rotaterd arr
    ay

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                rotated[j][n - 1 - i] = matrix[i][j];
            }
        }
        return rotated;
    }
```

Time Complexity: O(n²)
Space Complexity: O(n²)

Optimal: Transpose + Reverse Rows

transpose the matrix and the reverse it rows for row i start swaping from column j=i+1; half past hi run krns loop ka

```
void rotateMatrixOptimal(vector<vector<int>>& matrix) {
      int n = matrix.size();
      f

      for (int i = 0; i < n; i++) {
          for (int j = i + 1; j < n; j++) {
              swap(matrix[i][j], matrix[j][i]);
          }
      }

      for (int i = 0; i < n; i++) {
          reverse(matrix[i].begin(), matrix[i].end());
      }
  }
```

Time Complexity: O(n^2)
Space Complexity: O(1)

27. Spiral Traversal of Matrix

Problem

Print the matrix in spiral order.

Optimal: Boundary Traversal

```
vector<int> spiralOrder(vector<vector<int>>& matrix) {
      vector<int> ans;
      int n = matrix.size(), m = matrix[0].size();
      int top = 0, left = 0;
      int bottom = n - 1, right = m - 1;

      while (top <= bottom && left <= right) {
          for (int i = left; i <= right; i++) ans.push_back(matrix[top][i]);
          top++;

          for (int i = top; i <= bottom; i++) ans.push_back(matrix[i][right]);
          right--;

          if (top <= bottom) {
              for (int i = right; i >= left; i--) ans.push_back(matrix[bottom][i]);
              bottom--;
          }

          if (left <= right) {
              for (int i = bottom; i >= top; i--) ans.push_back(matrix[i][left]);
              left++;
          }
      }
      return ans;
  }
```

Time Complexity: O(n × m)
Space Complexity: O(1), excluding answer

28. Count Subarrays with Given Sum K

Problem

Count the number of subarrays whose sum is exactly `k`.

Brute Force: Generate Subarrays

```
int countSubarraysSumBrute(vector<int>& arr, int k) {
      int count = 0;

      for (int i = 0; i < arr.size(); i++) {
          int sum = 0;
          for (int j = i; j < arr.size(); j++) {
              for( int  k1=i;k1<=j; k1++){
                  sum += arr[k1];
              if (sum == k) count++;

              }

          }
      }
      return count;
  }
```

Time Complexity: O(n^3)
Space Complexity: O(1)

Optimal: Prefix Sum Frequency

If current prefix sum is `sum`, we need an old prefix sum `sum - k`.

```
int countSubarraysSumOptimal(vector<int>& arr, int k) {
      unordered_map<int, int> mp;
      mp[0] = 1;/subaray of sum o is always exist

      int prefixSum = 0;
      int count = 0;

      for (int x : arr) {
          prefixSum += x;
          int remove = prefixSum - k;
          count += mp[remove];
          mp[prefixSum]++;
      }
      return count;
  }
```

Time Complexity: O(nlogn) due to map Space Complexity: O(n)

Arrays Part 3: Hard and Final Problems

Use this header for all C++ snippets:

```
#include <bits/stdc++.h>
  using namespace std;
```

29. Pascal's Triangle

Problem

Pascal's Triangle has each value equal to the sum of the two values above it.

Type 1: Find Element at Row r and Column c

Use `nCr`, where answer is `C(r - 1, c - 1)` for 1-based indexing.

```
long long nCr(int n, int r) {
      long long ans = 1;
      for (int i = 0; i < r; i++) {
          ans = ans * (n - i);
          ans = ans / (i + 1);
      }
      return ans;
  }

  long long pascalElement(int row, int col) {
      return nCr(row - 1, col - 1);
  }
```

Time Complexity: O(c)
Space Complexity: O(1)

Type 2: Print nth Row

brute force

```
for( int col=1; col<=row ;i++){
      return nCr(row - 1, col -1);

  }
```

optimal

```
vector<long long> pascalRow(int row) {// roe is one based index
      vector<long long> ans;
      long long value = 1;
      ans.push_back(value);

      for (int i = 1; i < row; i++) {
          value = value * (row - i);
          value = value / i;
          ans.push_back(value);
      }
      return ans;
  }
```

Time Complexity: O(row)
Space Complexity: O(row)

Type 3: Print Whole Triangle

brute force

```
vector<vector<int>>ans;
  for( int row=1;row<=n; row++){
      vector< int> temp;


  for( int col=1; col<=row ;i++){
      temp.push_back( nCr(row - 1, col -1));

  }
  ans.push_back(temp);
  }
```

```
vector<vector<long long>> pascalTriangle(int n) {
      vector<vector<long long>> triangle;

      for (int row = 1; row <= n; row++) {
          triangle.push_back(pascalRow(row));
      }
      return triangle;
  }
```

Time Complexity: O(n²)
Space Complexity: O(n²)

30. Majority Element Greater than n/3

Problem

Find all elements that appear more than `n/3` times. There can be at most two such elements.

Brute Force: Count Every Element

```
vector<int> majorityN3Brute(vector<int>& arr) {
      vector<int> ans;
      int n = arr.size();

  for (int i = 0; i < n; i++) {
       if (ans.empty() || ans[0] != arr[i]){//arr[i]is previously not a part of my answ
  er so i need to check it;
              int count = 0;
            for (int j = 0; j < n; j++) {
              if (arr[j] == arr[i]) count++;
             }

          if (count > n / 3) ans.push_back(arr[i]);
          if (ans.size() == 2) break;

          }


      }

      sort(ans.begin(), ans.end());
      return ans;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1), excluding answer

Better: Hash Map

```
vector<int> majorityN3Better(vector<int>& arr) {
      unordered_map<int, int> freq;
      vector<int> ans;
      int n = arr.size();

      for (int x : arr) {
          freq[x]++;

      }
      for( auto it:fre){
          if(it.second>n/3){
              ans.push_abck( it.first);
          }
      }

      sort(ans.begin(), ans.end());
      return ans;
  }
```

is O(n)+ hash map ko iterate krna pda..

Better: Hash Map

```
vector<int> majorityN3Better(vector<int>& arr) {
      unordered_map<int, int> freq;
      vector<int> ans;
      int n = arr.size();

      for (int x : arr) {
          freq[x]++;
          if (freq[x] == n / 3 + 1) ans.push_back(x);
      }

      sort(ans.begin(), ans.end());
      return ans;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

Optimal: Extended Moore's Voting Algorithm

Keep two candidates because only two elements can appear more than `n/3` times.

```
vector<int> majorityN3Optimal(vector<int>& arr) {
      int count1 = 0, count2 = 0;
      int element1 = INT_MIN, element2 = INT_MIN;

      for (int x : arr) {
          if (count1 == 0 && x != element2) {//both of them must hold diffrent number t
  hats why i need toi check before assigning that it should not be equal to othet eleme
  nt
              element1 = x;
              count1 = 1;
          } else if (count2 == 0 && x != element1) {
              //both of them must hold diffrent number thats why i need toi check befor
  e assigning that it should not be equal to othet element
              element2 = x;
              count2 = 1;
          } else if (x == element1) {
              count1++;
          } else if (x == element2) {
              count2++;
          } else {
              count1--;
              count2--;
          }
      }

      count1 = 0;
      count2 = 0;
      for (int x : arr) {
          if (x == element1) count1++;
          else if (x == element2) count2++;
      }

      vector<int> ans;
      int mini = arr.size() / 3 + 1;
      if (count1 >= mini) ans.push_back(element1);
      if (count2 >= mini) ans.push_back(element2);

      sort(ans.begin(), ans.end());
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(1), excluding answer

31. 3 Sum

Problem

Find all unique triplets whose sum is `0`.

Brute Force: Three Loops

```
vector<vector<int>> threeSumBrute(vector<int>& arr) {
      set<vector<int>> st;
      int n = arr.size();

      for (int i = 0; i < n; i++) {
          for (int j = i + 1; j < n; j++) {
              for (int k = j + 1; k < n; k++) {
                  if (arr[i] + arr[j] + arr[k] == 0) {
                      vector<int> temp = {arr[i], arr[j], arr[k]};
                      sort(temp.begin(), temp.end());
                      st.insert(temp);
                  }
              }
          }
      }

      return vector<vector<int>>(st.begin(), st.end());
  }
```

Time Complexity: O(n^3 × log uniqueTriplets)
Space Complexity: O(uniqueTriplets)

Better: Hash Set for Third Element

Fix two elements using loops and search the third in a set.

```
vector<vector<int>> threeSumBetter(vector<int>& arr) {
      set<vector<int>> st;
      int n = arr.size();

      for (int i = 0; i < n; i++) {
          unordered_set<int> hashSet;//hash eill only store elemens b/w index i and j
  ie( i+1, to j-1)

          for (int j = i + 1; j < n; j++) {
              int third = -(arr[i] + arr[j]);

              if (hashSet.count(third)) {
                  vector<int> temp = {arr[i], arr[j], third};
                  sort(temp.begin(), temp.end());
                  st.insert(temp);
              }
              hashSet.insert(arr[j]);
          }
      }

      return vector<vector<int>>(st.begin(), st.end());
  }
```

Time Complexity: O(n² × log uniqueTriplets)
Space Complexity: O(n) + O(uniqueTriplets)

Optimal: Sort + Two Pointers

```
vector<vector<int>> threeSumOptimal(vector<int>& arr) {
      vector<vector<int>> ans;
      sort(arr.begin(), arr.end());
      int n = arr.size();
      //i ko fir rakhenge  j ko i+1 se inialtize krenge  aur k ko  n-1 se

      for (int i = 0; i < n; i++) {
          if (i > 0 && arr[i] == arr[i - 1]) continue;
          //ye sirf check krrne le kiye h ki same tpit na aa jaye isliye

          int j = i + 1;
          int k = n - 1;

          while (j < k) {
              int sum = arr[i] + arr[j] + arr[k];

              if (sum < 0) {
                  j++;
              } else if (sum > 0) {
                  k--;
              } else {
                  ans.push_back({arr[i], arr[j], arr[k]});
                  j++;
                  k--;

                  while (j < k && arr[j] == arr[j - 1]) j++;//same wale in j nhi honi c
  haihe second triplet chek krnw ke liye
                  while (j < k && arr[k] == arr[k + 1]) k--;//same wale in j nhi honi c
  haihe second triplet chek krnw ke liye
              }
          }
      }
      return ans;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: O(1), excluding answer

32. 4 Sum

Problem

Find all unique quadruplets whose sum equals the target.

Brute Force: Four Loops

```
vector<vector<int>> fourSumBrute(vector<int>& arr, int target) {
      set<vector<int>> st;
      int n = arr.size();

      for (int i = 0; i < n; i++) {
          for (int j = i + 1; j < n; j++) {
              for (int k = j + 1; k < n; k++) {
                  for (int l = k + 1; l < n; l++) {
                      long long sum = 1LL * arr[i] + arr[j] + arr[k] + arr[l];
                      if (sum == target) {
                          vector<int> temp = {arr[i], arr[j], arr[k], arr[l]};
                          sort(temp.begin(), temp.end());
                          st.insert(temp);
                      }
                  }
              }
          }
      }

      return vector<vector<int>>(st.begin(), st.end());
  }
```

Time Complexity: O(n^4 × log uniqueQuadruplets)
Space Complexity: O(uniqueQuadruplets)

Better: Hash Set for Fourth Element

```
vector<vector<int>> fourSumBetter(vector<int>& arr, int target) {
      set<vector<int>> st;
      int n = arr.size();

      for (int i = 0; i < n; i++) {
          for (int j = i + 1; j < n; j++) {
              unordered_set<long long> hashSet;// hasdh map store krega j aur ke beacch
  e ke eleemnts

              for (int k = j + 1; k < n; k++) {
                  long long sum = 1LL * arr[i] + arr[j] + arr[k];
                  long long fourth = target - sum;

                  if (hashSet.count(fourth)) {
                      vector<int> temp = {arr[i], arr[j], arr[k], (int)fourth};
                      sort(temp.begin(), temp.end());
                      st.insert(temp);
                  }
                  hashSet.insert(arr[k]);
              }
          }
      }

      return vector<vector<int>>(st.begin(), st.end());
  }
```

Time Complexity: O(n³ × log uniqueQuadruplets)
Space Complexity: O(n) + O(uniqueQuadruplets)

Optimal: Sort + Two Pointers

```
vector<vector<int>> fourSumOptimal(vector<int>& arr, int target) {
      vector<vector<int>> ans;
      sort(arr.begin(), arr.end());
      int n = arr.size();

      for (int i = 0; i < n; i++) {
          if (i > 0 && arr[i] == arr[i - 1]) continue;

          for (int j = i + 1; j < n; j++) {
              if (j > i + 1 && arr[j] == arr[j - 1]) continue;

              int k = j + 1;
              int l = n - 1;

              while (k < l) {
                  long long sum = 1LL * arr[i] + arr[j] + arr[k] + arr[l];

                  if (sum == target) {
                      ans.push_back({arr[i], arr[j], arr[k], arr[l]});
                      k++;
                      l--;

                      while (k < l && arr[k] == arr[k - 1]) k++;
                      while (k < l && arr[l] == arr[l + 1]) l--;
                  } else if (sum < target) {
                      k++;
                  } else {
                      l--;
                  }
              }
          }
      }
      return ans;
  }
```

Time Complexity: O(n³)
Space Complexity: O(1), excluding answer

33. Largest Subarray with 0 Sum

Problem

Find the length of the longest subarray whose sum is `0`.

Brute Force: All Subarrays

```
int longestZeroSumBrute(vector<int>& arr) {
      int maxLen = 0;

      for (int i = 0; i < arr.size(); i++) {
          int sum = 0;
          for (int j = i; j < arr.size(); j++) {
              sum += arr[j];
              if (sum == 0) maxLen = max(maxLen, j - i + 1);
          }
      }
      return maxLen;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Optimal: Prefix Sum First Index

If the same prefix sum appears again, the subarray between them has sum `0`.

```
int longestZeroSumOptimal(vector<int>& arr) {
      unordered_map<int, int> firstIndex;
      int sum = 0;
      int maxLen = 0;

      for (int i = 0; i < arr.size(); i++) {
          sum += arr[i];

          if (sum == 0) maxLen = i + 1;
          else if (firstIndex.count(sum)) maxLen = max(maxLen, i - firstIndex[sum]);
          else firstIndex[sum] = i;
      }
      return maxLen;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

34. Count Subarrays with Given XOR K

Problem

Count subarrays whose XOR is equal to `k`.

Brute Force: All Subarrays

```
int countXorBrute(vector<int>& arr, int k) {
      int count = 0;

      for (int i = 0; i < arr.size(); i++) {
          int xr = 0;
          for (int j = i; j < arr.size(); j++) {
              xr ^= arr[j];
              if (xr == k) count++;
          }
      }
      return count;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)

Optimal: Prefix XOR Frequency

If `prefixXor ^ oldPrefix = k`, then `oldPrefix = prefixXor ^ k`.

```
int countXorOptimal(vector<int>& arr, int k) {
      unordered_map<int, int> mp;
      mp[0] = 1;

      int xr = 0;
      int count = 0;

      for (int x : arr) {
          xr ^= x;
          int need = xr ^ k;
          count += mp[need];
          mp[xr]++;
      }
      return count;
  }
```

Time Complexity: O(n) average
Space Complexity: O(n)

35. Merge Overlapping Intervals

Problem

Given intervals, merge all overlapping intervals.

Brute Force: Sort and Expand Each Interval

```
vector<vector<int>> mergeIntervalsBrute(vector<vector<int>>& intervals) {
      sort(intervals.begin(), intervals.end());
      vector<vector<int>> ans;

      for (int i = 0; i < intervals.size(); i++) {
          int start = intervals[i][0];
          int end = intervals[i][1];

          if (!ans.empty() && end <= ans.back()[1]) continue;

          for (int j = i + 1; j < intervals.size(); j++) {
              if (intervals[j][0] <= end) {
                  end = max(end, intervals[j][1]);
              } else {
                  break;
              }
          }

          ans.push_back({start, end});
      }
      return ans;
  }
```

Time Complexity: O(n log n) + O(2n)// beacusec every element is vivsired twice
Space Complexity: O(n), excluding answer depending on sort

Optimal: Sort and Merge Once

```
vector<vector<int>> mergeIntervalsOptimal(vector<vector<int>>& intervals) {
      sort(intervals.begin(), intervals.end());
      vector<vector<int>> ans;

      for (auto interval : intervals) {
          if (ans.empty() || interval[0] > ans.back()[1]) {
              ans.push_back(interval);
          } else {
              ans.back()[1] = max(ans.back()[1], interval[1]);
          }
      }
      return ans;
  }
```

Time Complexity: O(n log n)
Space Complexity: O(n) for answer

36. Merge Two Sorted Arrays without Extra Space

Problem

Merge two sorted arrays without using an extra merged array.

Brute Force: Extra Array

```
void mergeSortedBrute(vector<long long>& a, vector<long long>& b) {
      vector<long long> temp;
      int i = 0, j = 0;

      while (i < a.size() && j < b.size()) {
          if (a[i] <= b[j]) temp.push_back(a[i++]);
          else temp.push_back(b[j++]);
      }

      while (i < a.size()) temp.push_back(a[i++]);
      while (j < b.size()) temp.push_back(b[j++]);

      for (int index = 0; index < temp.size(); index++) {
          if (index < a.size()) a[index] = temp[index];
          else b[index - a.size()] = temp[index];
      }
  }
```

Time Complexity: O(n + m)
Space Complexity: O(n + m)

Better: Swap and Sort

Place smaller values in first array and larger values in second array, then sort both.

```
void mergeSortedBetter(vector<long long>& a, vector<long long>& b) {
      //element from 0 to left are sorted in first aray any time
      //element from right to last are sorted in second array  aray any time
      int left = a.size() - 1;
      int right = 0;

      while (left >= 0 && right < b.size()) {
          if (a[left] > b[right]) {
              swap(a[left], b[right]);
              left--;
              right++;
          } else {
              break;
          }
      }

      sort(a.begin(), a.end());
      sort(b.begin(), b.end());
  }
```

Time Complexity: O(min(n, m)) + O(n log n) + O(m log m)
Space Complexity: O(1)

Optimal: Gap Method

Use Shell-sort style comparison gaps over the combined virtual array.

```
void swapIfGreater(vector<long long>& a, vector<long long>& b, int ind1, int ind2) {
      if (a[ind1] > b[ind2]) swap(a[ind1], b[ind2]);
  }

  void mergeSortedOptimal(vector<long long>& arr1, vector<long long>& arr2) {
      int n = arr1.size(), m = arr2.size();
      int len = n + m;
      int gap = (len / 2) + (len % 2);//ceil ((n+m)/2)

      while (gap > 0) {
          int left = 0;
          int right = left + gap;

          while (right < len) {
                  //arr1   aar2
              if (left < n && right >= n) {
                  swapIfGreater(arr1, arr2, left, right - n);
              }
              //arr2 arr2
              else if (left >= n) {
                  swapIfGreater(arr2, arr2, left - n, right - n);
              }
              //arr1 arr1
              else {
                  swapIfGreater(arr1, arr1, left, right);
              }

              left++;
              right++;
          }

          if (gap == 1) break;
          gap = (gap / 2) + (gap % 2);
      }
  }
```

Time Complexity: O((n + m) log(n + m))
Space Complexity: O(1)

37. Find Repeating and Missing Number

Problem

Array contains numbers from `1` to `n`. One number is missing and one number is repeated.

Brute Force: Count Every Number

```
pair<int, int> repeatMissingBrute(vector<int>& arr) {
      int n = arr.size();
      int repeating = -1, missing = -1;

      for (int num = 1; num <= n; num++) {
          int count = 0;
          for (int x : arr) {
              if (x == num) count++;
          }

          if (count == 2) repeating = num;
          else if (count == 0) missing = num;
          if(repeating!=-1 &&  missing!=-1){// means i got both of the elements
               break;
          }
      }
      return {repeating, missing};
  }
```

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Better: Hashing

```
pair<int, int> repeatMissingBetter(vector<int>& arr) {
      int n = arr.size();
      vector<int> hash(n + 1, 0);

      for (int x : arr) hash[x]++;

      int repeating = -1, missing = -1;
      for (int num = 1; num <= n; num++) {
          if (hash[num] == 2) repeating = num;
          else if (hash[num] == 0) missing = num;
            if(repeating!=-1 &&  missing!=-1){// means i got both of the elements
               break;
          }
      }
      return {repeating, missing};
  }
```

Time Complexity: O(n)
Space Complexity: O(n)

Optimal: Maths

Use sum and square-sum equations.

```
pair<int, int> repeatMissingOptimal(vector<int>& arr) {
      long long n = arr.size();
      long long sn = n * (n + 1) / 2;
      long long s2n = n * (n + 1) * (2 * n + 1) / 6;

      long long s = 0, s2 = 0;
      for (int x : arr) {
          s += x;
          s2 += 1LL * x * x;
      }

      long long val1 = s - sn;
      long long val2 = s2 - s2n;
      val2 = val2 / val1;

      long long repeating = (val1 + val2) / 2;
      long long missing = repeating - val1;

      return {(int)repeating, (int)missing};
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

38. Count Inversions

Problem

Count pairs `(i, j)` such that `i < j` and `arr[i] > arr[j]`.

Brute Force: Check All Pairs

```
long long countInversionsBrute(vector<int>& arr) {
      long long count = 0;

      for (int i = 0; i < arr.size(); i++) {
          for (int j = i + 1; j < arr.size(); j++) {
              if (arr[i] > arr[j]) count++;
          }
      }
      return count;
  }
```

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Optimal: Merge Sort

While merging, if `left[i] > right[j]`, then all remaining elements in the left half also form inversions.

```
long long mergeForInversions(vector<int>& arr, int low, int mid, int high) {
      vector<int> temp;
      int left = low;
      int right = mid + 1;
      long long count = 0;

      while (left <= mid && right <= high) {
          if (arr[left] <= arr[right]) {
              temp.push_back(arr[left++]);
          } else {
              temp.push_back(arr[right++]);
              count += mid - left + 1;
          }
      }

      while (left <= mid) temp.push_back(arr[left++]);
      while (right <= high) temp.push_back(arr[right++]);

      for (int i = low; i <= high; i++) arr[i] = temp[i - low];
      return count;
  }

  long long mergeSortInversions(vector<int>& arr, int low, int high) {
      if (low >= high) return 0;

      int mid = (low + high) / 2;
      long long count = 0;
      count += mergeSortInversions(arr, low, mid);
      count += mergeSortInversions(arr, mid + 1, high);
      count += mergeForInversions(arr, low, mid, high);
      return count;
  }

  long long countInversionsOptimal(vector<int> arr) {
      return mergeSortInversions(arr, 0, arr.size() - 1);
  }
```

Time Complexity: O(n log n)
Space Complexity: O(n)

39. Reverse Pairs

Problem

Count pairs `(i, j)` such that `i < j` and `arr[i] > 2 * arr[j]`.

Brute Force: Check All Pairs

```
int reversePairsBrute(vector<int>& arr) {
      int count = 0;

      for (int i = 0; i < arr.size(); i++) {
          for (int j = i + 1; j < arr.size(); j++) {
              if (1LL * arr[i] > 2LL * arr[j]) count++;
          }
      }
      return count;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Optimal: Merge Sort

Count valid cross pairs before merging the two sorted halves.

```
void mergeReversePairs(vector<int>& arr, int low, int mid, int high) {
      vector<int> temp;
      int left = low;
      int right = mid + 1;

      while (left <= mid && right <= high) {
          if (arr[left] <= arr[right]) temp.push_back(arr[left++]);
          else temp.push_back(arr[right++]);
      }

      while (left <= mid) temp.push_back(arr[left++]);
      while (right <= high) temp.push_back(arr[right++]);

      for (int i = low; i <= high; i++) arr[i] = temp[i - low];
  }

  int countPairs(vector<int>& arr, int low, int mid, int high) {
      int right = mid + 1;
      int count = 0;

      for (int i = low; i <= mid; i++) {
          while (right <= high && 1LL * arr[i] > 2LL * arr[right]) right++;
          count += right - (mid + 1);
      }
      return count;
  }

  int mergeSortReversePairs(vector<int>& arr, int low, int high) {
      if (low >= high) return 0;

      int mid = (low + high) / 2;
      int count = 0;
      count += mergeSortReversePairs(arr, low, mid);
      count += mergeSortReversePairs(arr, mid + 1, high);
      count += countPairs(arr, low, mid, high);
      mergeReversePairs(arr, low, mid, high);
      return count;
  }

  int reversePairsOptimal(vector<int> arr) {
      return mergeSortReversePairs(arr, 0, arr.size() - 1);
  }
```

Time Complexity: O(n log n)
Space Complexity: O(n)

40. Maximum Product Subarray

Problem

Find the maximum product of any contiguous subarray.

Brute Force: All Subarrays

```
long long maxProductBrute(vector<int>& arr) {
      long long maxi = LLONG_MIN;

      for (int i = 0; i < arr.size(); i++) {

          for (int j = i; j < arr.size(); j++) {
               long long product = 1;
              for( int k =i;k<=j; k++){
                  product *= arr[k];
                   maxi = max(maxi, product);

              }

          }
      }
      return maxi;
  }
```

Time Complexity: O(n^3)
Space Complexity: O(1)

Brute Force: All Subarrays

```
long long maxProductBrute(vector<int>& arr) {
      long long maxi = LLONG_MIN;

      for (int i = 0; i < arr.size(); i++) {
          long long product = 1;
          for (int j = i; j < arr.size(); j++) {
              product *= arr[j];
              maxi = max(maxi, product);
          }
      }
      return maxi;
  }
```

Time Complexity: O(n²)
Space Complexity: O(1)

Better: Prefix and Suffix Product

Zeros break product chains, so reset product to `1` when it becomes `0`.

```
long long maxProductBetter(vector<int>& arr) {
      long long prefix = 1;
      long long suffix = 1;
      long long ans = LLONG_MIN;
      int n = arr.size();

      for (int i = 0; i < n; i++) {
          if (prefix == 0) prefix = 1;
          if (suffix == 0) suffix = 1;

          prefix *= arr[i];
          suffix *= arr[n - i - 1];

          ans = max(ans, max(prefix, suffix));
      }
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)

Optimal: Track Maximum and Minimum Ending Here

A negative number can turn the minimum product into the maximum product.

```
long long maxProductOptimal(vector<int>& arr) {
      long long maxi = arr[0];
      long long mini = arr[0];
      long long ans = arr[0];

      for (int i = 1; i < arr.size(); i++) {
          long long x = arr[i];

          if (x < 0) swap(maxi, mini);

          maxi = max(x, maxi * x);
          mini = min(x, mini * x);

          ans = max(ans, maxi);
      }
      return ans;
  }
```

Time Complexity: O(n)
Space Complexity: O(1)